

# Logging

**Java Advanced Developer**

**StayAhead Training; Dec, 2025**

# About Logging

## Logging

- Logging is the writing to an output stream of data relating to system events
- Logging helps to identify bugs, misuse, and bottlenecks etc.
- Logging in Java requires a framework, either built-in or third-party

# Logging Frameworks

## Logging

`java.util.logging`



[LOGBACK]

Logging Abstractions



[SLF4J]



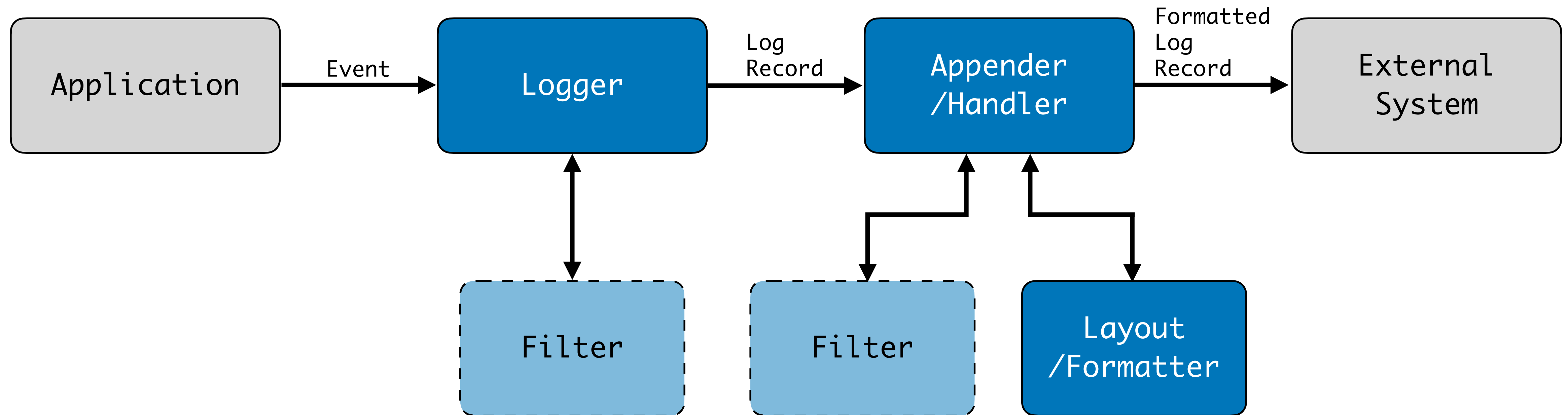
# Components

## Logging

| Component | Role                                                                                                     |
|-----------|----------------------------------------------------------------------------------------------------------|
| Logger    | Responsible for capturing events (Log Records) and passing them to an Appender                           |
| Appender  | Responsible for writing events to some external system; uses Layouts for formatting; AKA <b>Handlers</b> |
| Layout    | Responsible for converting and formatting event data; AKA <b>Formatters</b>                              |
| Filter    | Responsible for determining if an event is logged/appended (optional)                                    |

# Components

## Logging



# Configuration

## Logging

java.  
util.  
logging

Config. file:  
`<jdk_home>/conf/  
logging.properties`

Override by setting  
System property:  
`java.util.logging  
.config.file`

Log4J

Config. file:  
`log4j2.<format>`  
(on classpath)

Supports XML, JSON,  
and YAML

Logback

Config. file:  
`logback.<format>`  
(on classpath)

Supports XML and Groovy



# java.util.logging Config. Properties

## Logging

| Property          | Description                                                                 |
|-------------------|-----------------------------------------------------------------------------|
| handlers          | Whitespace/comma separated list of Appender class names for the root logger |
| <logger>.handlers | Ditto above but for the specified logger                                    |
| level             | The <u>minimum</u> level the root logger should log                         |
| <logger>.level    | Ditto above but for the specified logger                                    |
| <handler>.level   | The default log level for the given handler                                 |

# java.util.logging Config. Properties

## Logging

| Property                                           | Description                                                                           |
|----------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>&lt;handler&gt;.formatter</code>             | The formatter the given handler should use                                            |
| <code>java.util.logging.FileHandler.pattern</code> | The log file name pattern                                                             |
| <code>java.util.logging.FileHandler.limit</code>   | The approx. # of bytes to write before rotating to a new file                         |
| <code>java.util.logging.FileHandler.count</code>   | The number of log files to use on rotation                                            |
| <code>java.util.logging.FileHandler.append</code>  | Whether or not the handler should append to the existing file (assuming there is one) |



# Loggers

## Logging

- A Logger is an object that triggers log events
- A Logger is conventionally named after the class in which it's created, e.g.:

```
// assuming java.util.logging in com.example.logging.Main  
Logger logger = Logger.getLogger(Main.class.getName());
```

```
assert logger.getName().equals("com.example.logging.Main");
```

# Logger Hierarchy

## Logging

- A Logger named `com.example.logging.Main` implies a hierarchy of Loggers:
  - `""` (root)
  - `"com"`
  - `"com.example"`
  - `"com.example.logging"`
  - `"com.example.logging.Main"`
- Hierarchy Loggers are **not** created automatically (except root)
- Log events are forwarded to the Appender of the parent Logger by default

# Logger Hierarchy

## Logging

```
// logging.properties assuming java.util.logging
```

```
// a handler for the root Logger
```

```
handlers=java.util.logging.ConsoleHandler
```

```
// a handler for the com.example.logging Logger
```

```
com.example.logging.handlers=java.util.logging.ConsoleHandler
```

```
...
```

Will be logged twice - by the  
com.example.logging and root Loggers

```
Logger.getLogger("com.example.logging.Main").info("Hello");
```



# Logging Events

## Logging

```
// assuming java.util.logging  
logger.log(Level.INFO, "Request received");
```

```
// or...  
logger.info("Request received");
```

```
// drop all events below the given level  
logger.setLevel(Level.WARNING);
```

# Appenders/Handlers

## Logging

- An Appender is an object that writes log events to some external system
- An Appender uses a Layout/Formatter to format log events
- It is common to configure more than one Appender

```
// assuming java.util.logging  
logger.addHandler(new ConsoleHandler());
```

```
// or (more commonly) in the config. file:  
handlers=java.util.logging.ConsoleHandler ✓
```

# Appender Types

## Logging

| Appender type | Description                                                  |
|---------------|--------------------------------------------------------------|
| Console       | Writes log events to the console (stdout); often the default |
| File          | Writes log events to a file                                  |
| Syslog        | Sends log events to a syslog server (local or remote)        |
| Other         | E.g. rolling file, SMTP (email), failover etc.               |



# Layouts/Formatters

## Logging

- A Layout is an object that converts and formats log events
- `java.util.logging` provides `SimpleFormatter` and `XMLFormatter`
- Create a custom Layout by extending `Formatter`, e.g.:

```
public class MyCustomFormatter extends Formatter {  
    @Override  
    public String format(LogRecord record) {...}  
}
```

# Log Levels

## Logging

- `java.util.logging` levels listed by decreasing severity:
  - SEVERE
  - WARNING
  - INFO
  - CONFIG
  - FINE
  - FINER
  - FINEST
- See also ALL (log all regardless of level) and OFF (disables logging)